

Bayes Networks

CSC 480: Artificial Intelligence, Spring 2026
Professor Austin P. Wright: Course Notes

Abstract

We will cover the visual language of BayesNets (or Probabilistic Graphical Models), and how they are used to model complex probabilistic relationships.

I Bayes Networks

If we have a joint distribution for some set of random variables, we can answer any question about probabilities by inference through enumeration. However if there are n random variables, each with a sample space of size d , then the joint distribution table must have d^n entries, which in general we cannot hope to keep in memory for any nontrivial problem.

However, if we know something about the domain, and thus the conditional independence of the different variables, we can save an enormous amount of space by *factoring* the joint distribution in terms of conditional distributions. A Bayes Net (or Probabilistic Graphical Model) gives us a *language for specifying a conditional factorization structure of the joint distribution*.

Definition: Bayes Net

A Bayes Net consists of:

1. A *directed acyclic graph* of nodes, where each node corresponds to a single random variable (X).
2. A *conditional probability distribution* for each node $P(X \mid \text{parents}(X))$

Which together encode all of the information of the entire joint distribution (i.e. how the joint distribution factorizes in terms of conditional distributions)

Thus for the set of N random variables, the joint distribution can be written as

$$P(X_1, X_2, \dots, X_N) = \prod_{i=1}^N P(X_i \mid \text{parents}(X_i)) \tag{1}$$

The structure of a Bayes net graph encodes the conditional independence of the constituent variables. It is these conditional independence assumptions that allows the storage of multiple small probability tables instead of one huge table.

While encoding conditional independence and conditional distributions, remember that there is no sense in which edges in a Bayes Net must refer to a *causal* relationship, just that there is *some* relationship. Furthermore, these decisions about relationships (or lack thereof) are all *modeling choices* that enable real world problem solving. But since **all models are wrong**, since they are necessarily simplifications, we must very carefully interrogate these simplifications.

Contents

I	Bayes Networks	1
	Example	2
	Conditional Independence in	
	Bayes Nets	3
II	D-separation	4
	Common Structures (Triples) . . .	4
	D-separation Algorithm	4
III	Exact Inference: Variable	
	Elimination	6
	Ordering	6
	Example	7
IV	Approximate Inference	8
	Monte-Carlo Methods	8
	Prior Sampling	9
	Rejection Sampling	9
	Likelihood Weighting	9
	Gibbs Sampling	11

Discussion

Consider two models of the same two variables, rain and traffic. In one model we draw both variables as disconnected and independent. In the other we model rain as linked toward traffic. If an agent were to use either of those models as part of its probabilistic reasoning, which might you expect to be more useful? Also consider the other direction, with traffic linked toward rain, what is the difference? Think about *causality* and how it is useful but not enforced (especially if there may be missing variables).

Example

Consider an example of a Bayes net of slightly expanded problem from the previous lecture. Here the random variables are:

- B : Representing if a burglary occurs
- E : Representing if an earthquake occurs
- F : Representing if a fire occurs
- S : Representing if a smoke occurs
- A : Representing if a the alarm goes off
- J : Representing if Jeff calls 911
- M : Representing if Musty calls 911

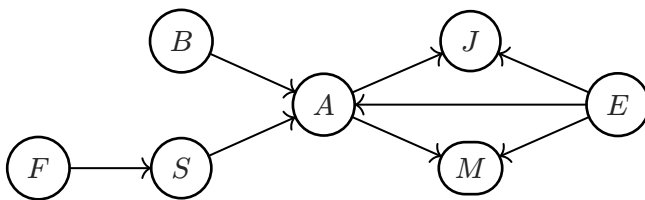


Figure 1: Example of Bayes Net

Discussion

Write out the joint distribution in terms of the conditional distributions according to the provided Bayes Net.

Conditional Independence in Bayes Nets

If the conditionals of a Bayes Net do in fact factorize the joint distribution, then the graph structure of the network allows us to make a number of claims of the conditional independence of the variables (this, essentially, is just the interpretation of *what we actually mean when we make a model using a Bayes net*). We have that:

1. Each node is conditionally independent of all ancestor nodes¹ in the graph, given all of its parents.
2. Each node is conditionally independent of *all other variables* given its *Markov Blanket*

¹Importantly, ancestors include *all non descendants* which could, in principle, be earlier than X in the DAG ordering.

Definition: Markov Blanket

A node's Markov blanket is the set of all of a node's parents, children, and other parents of it's children.

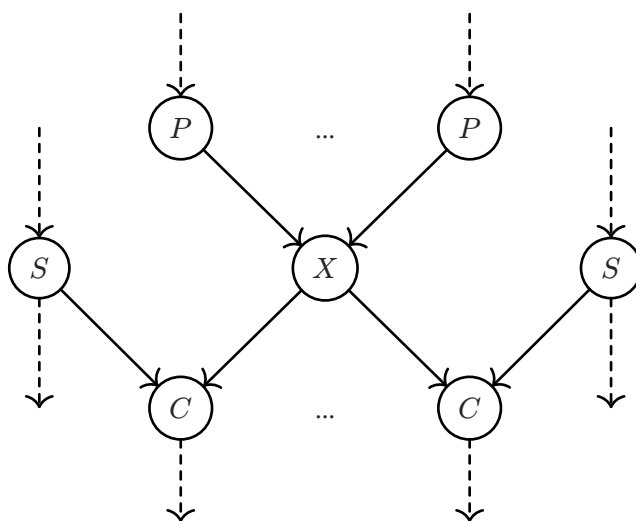


Figure 2: Markov Blanket for X

Discussion

Examine the earlier example of the alarm. Note how when writing the joint distribution in terms of the Bayes Net (i.e. $P(x \mid \text{parents})$), and then writing the joint in terms of the chain rule (i.e. $P(x \mid \text{parents, ancestors})$), the equation holds precisely as the conditional independence rules above hold.

II D-separation

D-separation (direct separation) is the process by which we can query the structure of a Bayes Net to answer questions about the conditional independence between arbitrary variables given arbitrary variables. Note that while the topology of the graph can communicate when *variables must be conditionally independent*, it does not enforce the other direction. It is possible that variables which the network considers possibly dependent may be, in reality, conditionally independent.

Common Structures (Triples)

While there is only two kinds of two variable relationships in a network (connected or disconnected), we can categorize a larger set of three node structures that will help us understand what we are looking at in a Bayes Net.



Figure 3: Causal Chain Structure

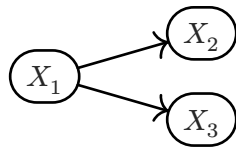


Figure 4: Common Cause Structure

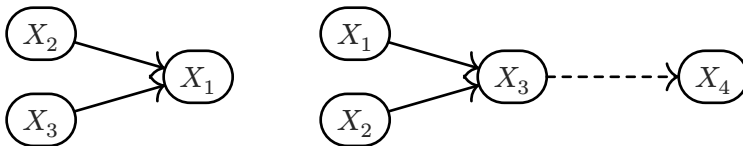


Figure 5: Common Effect Structure

D-separation Algorithm

1. Shade all observed nodes $\{Z_1, \dots, Z_k\}$ in the graph.
2. Enumerate all undirected paths from X to Y .
3. For each path:
 1. Decompose the path into triples.
 2. If all triples are open, the path d-connects X to Y .
4. If no path d-connects X and Y , then $X \perp\!\!\!\perp Y \mid \{Z_1, \dots, Z_k\}$ ².

²You can also derive these triples from the rules where a path is closed if either:

1. The arrows on the path meet either head-to-tail or tail-to-tail at a colored node
2. The arrows meet head-to-head at the node, and neither the node, nor any of its descendants is colored.

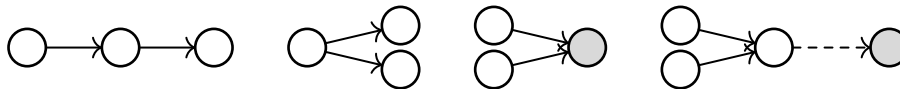


Figure 6: Open Triples

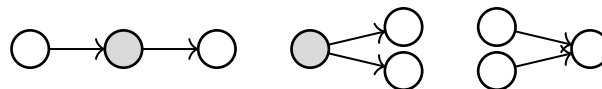
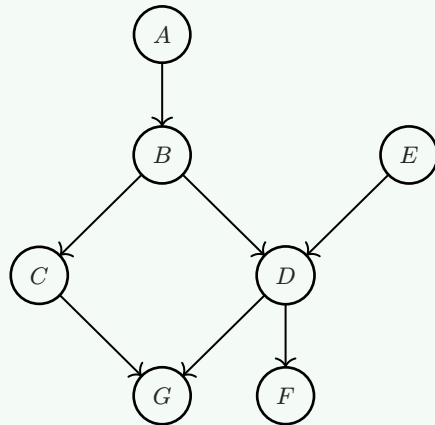


Figure 7: Closed Triples

Discussion

For the provided BayesNet, which of the following conditional independences *must* hold?

$$A \perp\!\!\!\perp F \mid D$$

$$A \perp\!\!\!\perp E$$

$$A \perp\!\!\!\perp E \mid D$$

$$A \perp\!\!\!\perp E \mid F$$

$$A \perp\!\!\!\perp E \mid D, B$$

$$A \perp\!\!\!\perp E \mid D, B$$

$$C \perp\!\!\!\perp D$$

$$C \perp\!\!\!\perp D \mid B$$

$$C \perp\!\!\!\perp D \mid B, G$$

(2)

III Exact Inference: Variable Elimination

The primary task our agents might have when using Bayes Nets to model the world is *inference*, that is finding some arbitrary new conditional probability from the joint (generally as a result of conditioning on new observation data). We already know how to do this by using the Bayes Net to create the joint distribution and then using inference by enumeration, but this is intractable. Luckily we can do better while maintaining an *exact* answer (i.e. always precisely the same probability as would be generated by full join inference), using a technique called *Variable Elimination*³.

Definition: Variable Elimination

To *eliminate* a variable X we simply:

1. Join all *factors* involving X
2. Sum out X

Where a *factor* is simply an *unnormalized probability* given by selecting entries from the conditional probability table consistent with the evidence.

³While this method is much more efficient than the exponential inference by enumeration, it is still NP-Hard and so while there are some problems for which it is tractable, often we will need to use approximate solutions which will be covered next

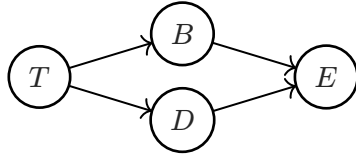
```
def eliminate(x: Query, e: Evidence, bn: BayesNet):
    factors = []
    for var in bn.Variables:
        factors = [MAKE_FACTOR(var,e)] + factors
        if is_hidden_variable(var):
            factors = SUM_OUT(var,factors)
    return NORMALIZE(PRODUCT(factors))
```

Ordering

Note that, like many of the algorithms in this class, there is a component where we need to iterate over the variables in the BayesNet in some non-deterministically defined order. It turns out that the order makes a massive impact on the efficiency of variable elimination, and heuristics are used here to choose what we think will be good orders. Such heuristics include *minimum degree* where we prioritize eliminating variables which results in constructing the smallest factor possible, and *minimum fill* which constructs an undirected graph of variable relations and eliminates the variable that would result in the fewest edges added after elimination.

Example

Suppose we have the model shown below. Where a wizard has an opportunity to take a golden statue from a suspicious pedestal in some ancient ruins. Let us say T is the random variable representing if the wizard takes the statue. Then B is a random variable representing if a boulder falls from the ceiling, D represents darts being shot from the walls, and E represents the wizard escaping the ruins. We then have the BayesNet below:



This gives us the conditional probability distributions:

$$P(T), P(B | T), P(D | T), P(E | B, D) \quad (3)$$

Suppose we want to calculate the probability that the wizard took the statue, given that it was able to escape: $P(T | e)$. Using variable elimination we don't have to calculate the whole 16 row joint, instead we can eliminate B then D which is done as follows:

1. Join (multiply) factors involving B , giving:

$$f_1(T, B, D, e) = P(B | T) \cdot P(e | B, D) \quad (4)$$

1. Sum out B from this new factor, leaving $f_2(T, D, e)$
2. Join factors involving D forming:

$$f_3(T, D, e) = P(D | T) \cdot f_2(T, D, e) \quad (5)$$

1. Sum out D , giving $f_4(T, e)$.
2. Join the remaining factors, which gives:

$$f_5(T, e) = f_4(T, e) \cdot P(T) \quad (6)$$

1. Once we have $f_5(T, e)$, we can find $P(T | e)$ by normalizing.

One thing that we can see is how variable elimination improves over inference by enumeration. In inference by enumeration we must first join *all* of the conditional distributions, then sum out all of the variables:

$$\sum_b \sum_d P(T) P(b | T) P(d | T) P(e | b, d) \quad (7)$$

But in variable elimination we are able to simply move the terms that are constant in a sum outside of the sum, reducing the size of the inner tables:

$$P(T) \sum_b P(b | T) \sum_d P(d | T) P(e | b, d) \quad (8)$$

IV Approximate Inference

When reasoning with Bayes Nets, our primary task is probabilistic inference, or updating and calculating probabilities in response to new observations to condition on. We can do this exactly by inference by enumeration (exponentially hard), or by variable elimination (NP-Hard). However, since the Bayes Net itself is ultimately just an approximation, perhaps we can find solutions that are approximate themselves with respect to the Bayes Net instead of needing to find the exact solution. We do this by simply counting samples from our Bayes Net conditional distribution, and counting up those samples, which ends up being massively more efficient than exact solutions in most contexts and frequently serves as a good enough approximation.⁴

Monte-Carlo Methods

Our core idea for finding an approximation of an inferred probability is to use a class of solution called a Monte-Carlo method⁵.

Monte Carlo methods take as the starting point that doing probabilistic inference is hard, because reasoning about the relations of information between whole probability distributions and transformations on probability distributions requires doing computational work for not just single values, but for the full sets of values that the probability distribution covers in the sample space. Instead what we can do is simply sample individual values from our distributions, do whatever transformations need to be done on those values, and then count up the samples to get estimates of the final distribution. The work now becomes much simpler, computation now is over concrete rather than distributional variables, and can scale to whatever degree of accuracy is needed by generating a higher number of samples, which in general may be much smaller than the exponential work needed to perform exact inference. Furthermore, samples can often be generated and processed massively in parallel.

Discussion

Recall the definition of what we think of as computationally hard, but not impossible, problems in NP⁶. Loosely, the class of NP is all problems for which we can check a *specific* solution easily, but it is hard to generate a *general* solution. Consider how the Monte-Carlo approach might relate to such problems, where we can easily generate and work with *samples* and count them up as an estimate of the general solution *distribution*.

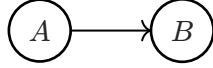
⁴We will discuss some of the situations where sampling either does not produce a good approximation or has other computational limitations, and how to address them. However I also want to point you towards other approximate inference techniques outside of the scope of the course that do not rely on sampling, such as *Variational Inference*, which is a core method for most contemporary deep learning and generative models that uses training parametrized approximations rather than sampling.

⁵In the literature you will also see reference to MCMC or Markov Chain Monte-Carlo which is the more well defined generalization of this kind of method. The approaches we will be discussing are special cases of MCMC, such as Gibbs sampling being a special case of the Metropolis-Hastings algorithm.

⁶Obviously assuming $P \neq NP$

Prior Sampling

The primary thing that makes Monte-Carlo methods possible for us, is that given a Bayes Net, we can easily generate samples. For instance, consider the following simple network.



For this network we must have the conditional distributions: $P(A)$, $P(B|A)$. From these distributions we can very easily generate a sample of the joint distribution by first sampling the unconditioned variable A from $P(A)$ yielding some value a , then using that sampled value to condition for a sample of B from $P(B | A = a)$ yielding b . This pair, (a, b) , is now a sample from the joint distribution $P(A, B)$, and with enough samples the count frequencies will converge to the true distribution (assuming our sampling is unbiased).

This is essentially all we need for Monte-Carlo approximate inference, the rest is just optimizations.

Rejection Sampling

The first optimization we can do is noticing that if we have some *evidence* that we want to condition our final distribution on, then samples that do not agree with the evidence don't help us at all. For instance, imagine in the previous example we wanted to query $P(B | A = a)$. Then every time we sampled a value $A \neq a$, the resultant joint from sampling $P(B|A)$ is wasted work. Therefore we can use *rejection sampling* to throw out any sample early once we learn that it is inconsistent with the conditioning of our query.

Likelihood Weighting

The next optimization we can notice is that, depending on the actual probability of the conditional distributions “upstream” in a network, some joint outcomes can end up being very rare. But if we are interested in inference related to those rare outcomes, this would mean having to generate a very large number of samples in order to get enough samples in our relevant “end state”.

A first crack at this might say, instead of sampling and then rejecting samples, if we have a fixed evidence condition, just set the value to that condition (i.e., just set $A = a$ and only generate samples for $P(B|a)$). In fact, this would work for our simple example. However, in general, this can sometimes result in biased estimates. We can see this by considering a set of sample variables $Z_1, \dots, Z_n$ and a set of evidence variables $E_1, \dots, E_m$. Then the joint distribution which we are trying to approximate is $P(Z_1, \dots, Z_n, E_1, \dots, E_m)$. However if we simply set the values of the evidence directly we have

$$P(Z_1, \dots, Z_n, E_1, \dots, E_m) = \prod_{i=1}^n P(Z_i | \text{Parents}(Z_i)) \quad (9)$$

Which is actually missing the sometimes important information of the conditional distributions of the evidence, $P(E_j \mid \text{Parents}(E_j))$. Think back to just applying Bayes rule, we cannot just throw out the evidence term $P(E)$, even if we actually know what the evidence is.

We can solve this problem through *likelihood weighting*, which works by setting a weight for each sample which is the probability of the observed evidence variables given the sample values. This way every conditional distribution contributes to the joint, where the weights serve to replace otherwise missing information. We can see how for a sample, the *weighted frequency* becomes the true joint probability:

$$P(z_1, \dots, z_n, e_1, \dots, e_m) = \left[\prod_{i=1}^n P(z_i \mid \text{Parents}(z_i)) \right] \cdot \left[\prod_{i=1}^m P(e_i \mid \text{Parents}(e_i)) \right] \quad (10)$$

The algorithm for likelihood weighting, informally, proceeds as:

1. Initialize weight of a sample to 1
2. For each of the variables in the Bayes Net in order, either:
 1. If it is not an evidence variable, simply sample the value based on the conditional distribution of the variable.
 2. If it is an evidence variable, set the value of the sample to the evidence value, and then multiply the weight of the sample by the conditional distribution of the evidence variable.
3. In the final inference probability aggregation calculation, weight each sample using its weight value.

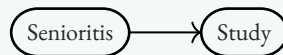
While this is much better than rejection sampling, we still have the issue of generating accurate estimates of small probabilities. Instead of simply throwing away a large number of samples, we end up where for those probabilities we still only have a small number of highly weighed samples which dominate the estimate and thus using an effectively small sample with higher variance.

Discussion

Recall the model of diagnosis for senioritis:

$$\begin{aligned} P(\text{senioritis}) &= 0.01 \\ P(\text{study} \mid \text{senioritis}) &= 0.1 \\ P(\text{study} \mid \neg \text{senioritis}) &= 0.9 \end{aligned} \quad (11)$$

We have the corresponding network now:



For the query, $P(\text{Senioritis} \mid \text{Study})$, if we generate 1000 samples, how many do we expect to have senioritis? What are their weights? What about samples without senioritis? What are their weights? When calculating the final probability, what is the expected “effective” sample size (which we can think of as the sum of the weights)?

Gibbs Sampling

The fourth and final approach we will cover to sample based approximate inference is fairly different to the other approaches, but frequently much more efficient for really complex interconnected networks. Instead of starting from the “top” of the network and sampling through, for Gibbs sampling we will do operations over the whole network in a more distributed style. This may allow us to have the influence of evidence “flow both ways” where in likelihood weighting the evidence influences choices of children, but does not effect the samples of parents even though the evidence does give us information about the parents. For Gibbs sampling we want to consider the evidence when we sample *every variable*.

To do this we start by setting all of the variables in the network to completely random values (entirely ignoring the conditional distributions). We then repeatedly choose one variable at a time (not in any kind of deterministic topological order), clear its current value, and resample the value given the current assigned values of its markov blanket. Note that this is *the whole markov blanket*, not just the parents, since we are treating all variables in the network as conditions, and thus we need to condition on the whole set of conditional variables to isolate the sampled variable from the rest of the network. Luckily this generalization of the provided Bayes Net conditionals can be calculated once before the sampling process using only the set of local conditional distributions of the blanket. This can be written as :

$$P(x_i \mid x_1, \dots, x_N) = \frac{1}{Z} p(x_i \mid \text{parents}(x_i)) \prod_{j \in \text{children}(x_i)} p(x_j \mid \text{parents}(x_j))$$

$$Z = \sum_{x_i} p(x_i \mid \text{parents}(x_i)) \prod_{j \in \text{children}(x_i)} p(x_j \mid \text{parents}(x_j)) \quad (12)$$

Over many iterations, our samples will eventually converge to the correct distribution⁷. We can see how evidence can be accounted for by simply setting the evidence variables to the evidence values and never selecting evidence variables to resample from. Since we are including all of the relevant conditional distributions at each resample we do not have the same problem as naive sampling from above (since for each actual resample we are treating the whole rest of the network as fixed conditional anyway.) You can also notice how each sample is built from the previous sample in a “chain”, which is *precisely* the chain in Markov Chain Monte Carlo methods of this this is an example.

⁷ Except in some pathological cases which can generally be addressed by performing Gibbs sampling multiple times with different initializations.

Note that there are a few important caveats to this distribution.

1. The resultant samples are *highly dependent* (since each sample will only differ from the previous sample in precisely one variable). This is sometimes addressed with *subsampling* where we only include every k^{th} sample in our final sample set.
2. It requires a “burn in” period where the initial samples are very unlikely due to the random initialization
3. Gibbs relies on being able to explore the whole sample space eventually (and in principle visit each node infinitely many times). Since the initialization is a random and unlikely value, and each update only updates a single variable,

if variables are strongly correlated enough to effectively “wall off” certain regions of the joint, convergence may be impossible or highly unlikely. This is frequently a consequence of the “curse of dimensionality”.

However, in the limit of a large number of samples⁸ (and frequently with the ability to throw out some predetermined amount of the initial samples) this still acts as a valid sampler.

⁸Of course, with such methods what constitutes a sufficiently large sample for representativeness and convergence is not generally known ahead of time and so may be an issue.

Discussion

Consider a two variable joint distribution where the probability distribution is uniform over a finite “checkerboard” in a continuous domain. What might be an issue with Gibbs sampling for this problem? Would likelihood weighting have the same problem?

Discussion

Consider a joint distribution over 100-bit vector where the probability distribution is uniform over all vectors except for the zero vector which has probability $\frac{1}{2}$. What happens with Gibbs sampling when we have a value in the nonzero region? What about the zero region? What does this imply about the effectiveness of Gibbs sampling in this context? Would likelihood weighting have the same problem?